Reinforcement Learning for Lifecycle Portfolio Allocation: From Exact Dynamic Programming to Function Approximation

Patrick Flanagan[1], Jack Zhang[1], and Jeffrey Xue[1]

[1]Stanford University, CME 241 — Reinforcement Learning for Stochastic Control in Finance (Winter 2026)

March 20, 2026

Abstract

We study finite-horizon lifecycle consumption and portfolio choice with stochastic labor income and market regimes, maximizing discounted constant relative risk aversion (CRRA) utility of consumption plus a terminal bequest. A tractable discrete-state formulation is solved exactly by backward dynamic programming (DP), validating objectives and baselines. A scaled "realistic" parameterization exhibits a roughly 119× increase in DP table size, motivating reinforcement learning (RL). We implement a Gymnasium-style simulator and four agents: linear Q-learning, SARSA with linear function approximation, Dueling Deep Q-Networks (DQN), and Proximal Policy Optimization (PPO). Engineering fixes—consistent penalization of zero consumption, correct training dispatch for on-line versus replay-based updates, randomized initial states, and architecture choices—are essential for learning non-degenerate, state-dependent policies. Monte Carlo evaluation on 2,000 simulated lifetimes shows all RL methods outperform static 60/40, equal-weight, and glidepath rules on expected utility, with Dueling DQN achieving the highest mean utility among RL agents while linear methods offer interpretability and orders-of-magnitude faster training.

Keywords: lifecycle portfolio choice; dynamic programming; reinforcement learning; CRRA utility; SARSA; DQN; PPO

1 Introduction

Households approaching retirement face a joint problem: how much to consume each period versus save, and how to allocate savings across risky and safe assets, while income and investment opportunities evolve stochastically. Target-date funds and rules of thumb (e.g., static stock–bond mixes or deterministic glidepaths) are widely used but need not optimize a well-specified welfare objective.

This paper documents a two-stage computational project. **Phase 2** embeds the problem in a finite Markov decision process (MDP) with a coarse wealth grid and solves it *exactly* by backward induction, comparing the optimal policy to simple benchmarks. **Phase 3** enlarges the horizon, the number of income and regime states, and the fineness of the action grid; exact tabular DP becomes impractical. We instead learn policies with RL, using function approximation over a low-dimensional state feature vector and a discrete joint action space for consumption fraction and portfolio weights.

Our contributions are: (i) a reproducible DP baseline and curse-of-dimensionality accounting; (ii) a unified open-source environment (`PortfolioEnv`) with economically interpretable rewards $\beta^t u(c_t)$; (iii) four RL algorithms spanning off-policy and on-policy, linear and deep; and (iv) an explicit discussion of implementation pitfalls (reward bugs, training dispatch errors, evaluation metrics) that dominate short-horizon applied RL projects.

2 Related work

Lifecycle models with portfolio choice are central in macro-finance [Samuelson, 1969, Merton, 1969]. Exact solutions are typically available only in continuous-time special cases; realistic models require numerical methods. Approximate dynamic programming and fitted value iteration are standard in economics [Judd, 1998]. Deep RL has been applied to portfolio management and related control problems [Jiang et al., 2017]; our setting emphasizes consumption–portfolio interaction and finite horizons with explicit discounting in the reward. Algorithmically we build on Q-learning and SARSA [Sutton and Barto, 2018], DQN and Dueling architectures [Mnih et al., 2015, Wang et al., 2016], and PPO [Schulman et al., 2017].

3 Problem formulation

3.1 MDP structure

Let time $t \in \{0, \ldots, T-1\}$. The state is $s_t = (t, W_t, y_t, z_t)$: calendar time, wealth $W_t \geq 0$, discrete income state y_t, and discrete market regime z_t. The action is a pair $(\alpha_t, \mathbf{w}_t)$: consumption fraction $\alpha_t \in [0, 1]$ (chosen from a finite grid) and portfolio weights $\mathbf{w}_t$ over stocks, bonds, and cash (simplex, discretized). Consumption is $c_t = \alpha_t W_t$; savings are $(1-\alpha_t)W_t$. Gross asset returns $\mathbf{R}_t$ are drawn from a finite set whose distribution depends on z_t. Next-period wealth is

$$W_{t+1} = \big(W_t - c_t\big)\,(\mathbf{w}_t^\top \mathbf{R}_t) + \text{income}(y_{t+1}), \tag{1}$$

with y_{t+1} and z_{t+1} following Markov transitions. The objective is

$$\max_{\pi}\ \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \beta^t u(c_t) + \beta^T u(W_T)\right], \tag{2}$$

with CRRA utility $u(c) = c^{1-\gamma}/(1-\gamma)$, $\gamma = 2$, so $u(c) = -1/c$ for $c > 0$. Near-zero consumption is penalized by a large negative surrogate in code for numerical stability (aligned with DP treating zero consumption as infeasible).

3.2 Phase 2: exact DP (small scale)

We discretize wealth on 121 grid points, use two income and two regime states, $T = 25$, $\beta = 0.96$, $\gamma = 2$, 11 consumption fractions $(0\%, 2\%, \ldots, 20\%)$, and 15 allocations (25% increments). There are 165 actions per state and about 2.0×10^6 DP cells per full backward pass; the solver runs in under one second. Terminal values are $V_T(W) = u(W)$; for $t < T$ we maximize over actions and expectation over income, regime, and return shocks, with interpolation on the wealth grid where needed.

3.3 Phase 3: scaled MDP and RL interface

Phase 3 uses $T = 40$, three income levels $\{3, 10, 25\}$, three regimes {bear, normal, bull}, transition matrices as in the codebase, three return scenarios per regime, 11 consumption fractions, and 21 three-asset allocations (20% increments), yielding **231** atomic actions. Wealth is treated as continuous within $[W_{\min}, W_{\max}]$ with $W_{\min} = 0.01$, $W_{\max} = 800$, and initial evaluation wealth $W_0 = 100$.

The environment maps the latent state to an **eight-dimensional** feature vector: normalized time t/T, normalized log wealth $\log W / \log W_{\max}$, one-hot(y) (3), and one-hot(z) (3). The step reward is $r_t = \beta^t u(c_t)$; at termination an additional $\beta^T u(W_T)$ is added. RL discount γ_{RL} is set to 1 so that undiscounted return equals the economic objective; discounting is entirely in the reward.

Training uses **randomized resets**: log-uniform wealth in $[10,\, 0.6\, W_{\max}]$, uniform random income and regime. This widens state coverage and is critical for state-dependent policies.

4 Algorithms

4.1 Linear Q-learning

We approximate $Q(s,a) \approx \mathbf{w}_a^\top \boldsymbol{\phi}(s)$ with shared features $\boldsymbol{\phi}(s) \in \mathbb{R}^{24}$: raw 8-D state, polynomials (t^2, w^2, tw), interactions of normalized time and wealth with each of six categorical indicators, and a bias. There are $231 \times 24 = 5544$ scalar weights. Updates are semi-gradient Q-learning with ε-greedy exploration. Hyperparameters: learning rate $\alpha = 10^{-5}$, $\gamma_{\mathrm{RL}} = 1$, ε linearly from 1 to 0.05 over 8,000 episodes, 15,000 episodes total.

4.2 SARSA

The same linear architecture is trained on-policy: the TD target uses $Q(s', a')$ where a' is the action that will be executed, enforced via a cached next-action mechanism. TD errors are clipped to $[-10, 10]$ for stability under large negative utilities.

4.3 Dueling DQN

A neural network with trunk $8 \to 256 \to 256$ (tanh) feeds a value head $256 \to 1$ and an advantage head $256 \to 231$, combined as $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$ [Wang et al., 2016]. Experience replay (10^5 transitions), target network updates every 200 steps, Huber loss, Adam learning rate 3×10^{-4}, batch size 128, gradient clip norm 10.

4.4 PPO

Separate actor and critic MLPs ($8 \to 256 \to 256 \to$ logits or scalar value), orthogonal initialization, rollout length 2048, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, entropy coefficient 0.05, 10 epochs per rollout, minibatches of 128. Stochastic policy with scheduled exploration decay.

4.5 Training infrastructure

Agents declare `uses_replay`: DQN and PPO call `store`/`train_step`; linear Q and SARSA call `update` each step. This avoids a subtle bug where a base-class `store` stub caused linear methods to never update.

5 Implementation issues and remedies

Zero-consumption exploit. Returning reward zero when $c = 0$ made "never consume" optimal because $u(c) < 0$ for all $c > 0$ under $\gamma = 2$. Rewards now always use the CRRA-based penalty for negligible consumption.

Evaluation utility. Monte Carlo utility accumulation must apply the same penalty; skipping zero-consumption periods had inflated reported utility for degenerate policies.

Architectures. PPO benefited from decoupled actor–critic networks; DQN benefited from the dueling head and larger width (256). Randomized initial states dominated other changes for visible state dependence in policies.

6 Experiments

We train each agent for 15,000 episodes. Evaluation uses 2,000 simulated paths starting at wealth $W_0 = 100$, medium income, and normal regime. Baselines are static 60/40 stocks/bonds, equal thirds, and a linear stock glidepath 80% $\rightarrow$ 20% with the remainder in bonds. Table 1 reports mean discounted CRRA utility and terminal wealth moments; Table 2 summarizes risk-style metrics from the same simulations.

Table 1: Monte Carlo evaluation (2,000 paths). Utility is expected total discounted CRRA objective; wealth statistics are terminal W_T.

Strategy	$\mathbb{E}[U]$	$\mathbb{E}[W_T]$	Med$[W_T]$	Std$[W_T]$	$P_{10}[W_T]$	$P_{90}[W_T]$
DQN	-1.54	115.3	107.2	51.8	57.9	185.5
Linear Q	-1.59	96.0	86.6	33.8	59.8	144.2
SARSA	-1.88	99.1	93.3	35.3	58.1	149.9
PPO	-1.95	91.0	81.2	43.2	44.8	149.8
Static 60/40	-2.63	487.3	470.1	166.1	281.0	747.3
Glidepath	-2.63	489.1	474.1	140.0	319.7	696.5
Equal 1/3	-2.75	407.8	396.1	112.4	270.5	564.4

Table 2: Risk-style metrics (same simulations). Sharpe(C) denotes a consumption-based Sharpe measure computed in the project pipeline.

Strategy	Max drawdown (%)	Sharpe(C)	min W_T
DQN	60.4	26.53	18.3
Linear Q	54.0	18.30	33.2
SARSA	57.1	10.25	24.9
PPO	66.1	12.58	20.5
Static 60/40	23.8	2.65	98.4

Figure 1: Training diagnostics (100-episode smoothed returns, running averages, and loss for applicable agents). PPO typically moves fastest in return space; DQN loss decreases as replay fills.

Phase 2 DP on the small grid yields mean utility about -0.95 and mean terminal wealth ≈ 88.5 on comparable baseline simulations; see [**?**]. Scaling to Phase 3 changes absolute utility levels but preserves the ranking insight: **optimizing consumption timing** under CRRA need not maximize terminal wealth. RL agents rank above baselines on $\mathbb{E}[U]$ while exhibiting lower $\mathbb{E}[W_T]$.

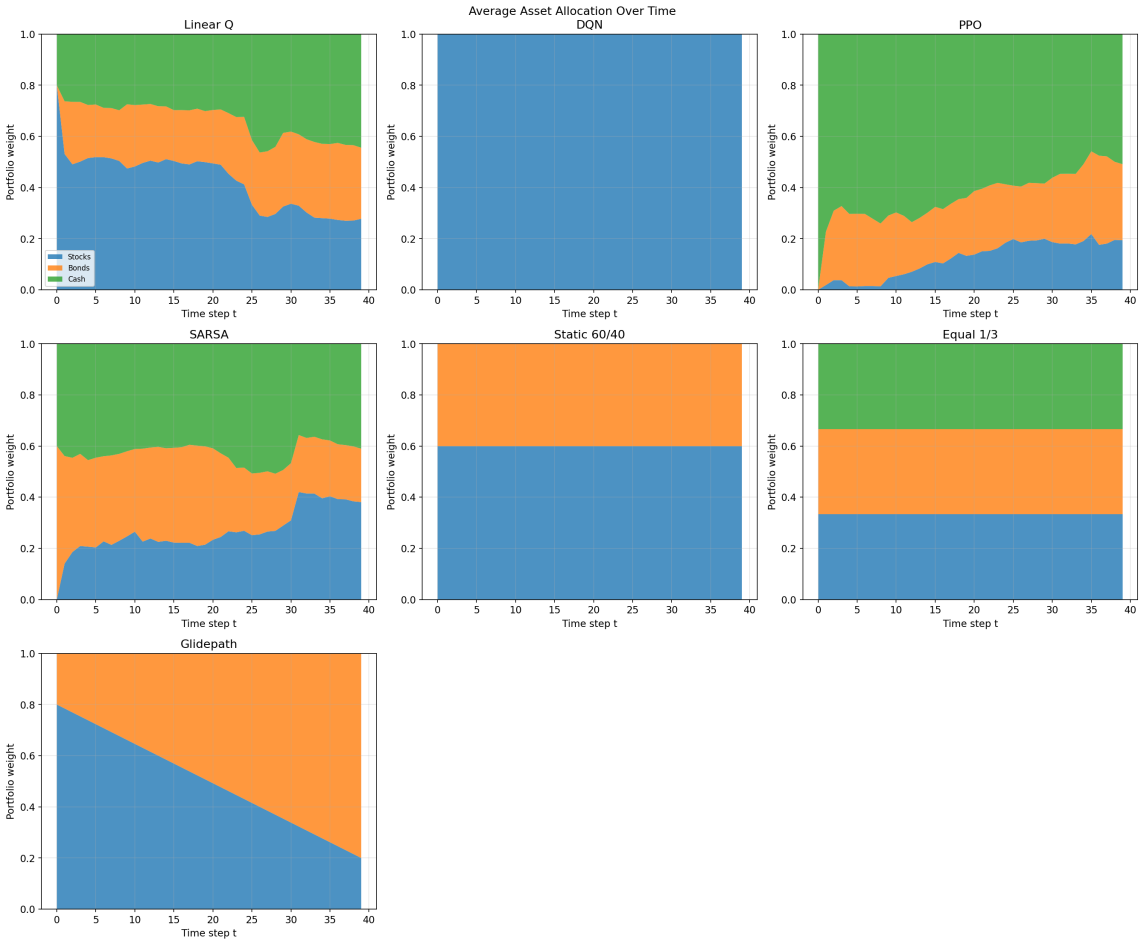

Figure 2: Average portfolio weights over the horizon for RL agents and baselines (stocks, bonds, cash). Linear Q exhibits a smooth de-risking; DQN concentrates in stocks; PPO and SARSA use substantial cash early before reallocating.

7 Discussion

Linear Q and SARSA produce interpretable, highly state-dependent heatmaps when consumption and stock shares are plotted against time and wealth; explicit feature interactions help. DQN achieves the best mean utility here with a nearly stationary aggressive policy (high equity, high consumption grid point), which is consistent with a long horizon and positive equity premium in the simulator. PPO shows strong time structure (e.g., cash-heavy early periods) but underperforms DQN on $\mathbb{E}[U]$ in our runs. Off-policy value methods outperform on-policy counterparts (Linear Q vs SARSA; DQN vs PPO) on this metric, though SARSA offers conservative tail behavior (higher $\min W_T$ than DQN).

Limitations. The joint discrete action space is inefficient; factored actions or continuous control would improve exploration. Regimes and income are stylized; transaction costs, taxes, and empirical calibration are omitted. Training seed and agent order affect reproducibility when a shared random stream is used across sequential training runs.

8 Conclusion

Exact DP validates the economic problem and clarifies objectives, but a modest increase in problem size makes full tabulation prohibitive. With careful reward design and training infrastructure, four standard RL algorithms learn policies that dominate common investment heuristics on discounted CRRA welfare. The gap between terminal wealth and utility underscores the importance of aligning computational objectives with household welfare. Future work includes continuous actions, larger asset menus, and estimation from market and panel data.

Acknowledgments

Course project for CME 241 (Stanford, Winter 2026). Code and configuration are available in the repository under `project/phase3/`.

References

Judd, K. L. (1998). *Numerical Methods in Economics.* MIT Press.

Jiang, Z., Liang, J., and Li, L. (2017). Cryptocurrency portfolio management with deep reinforcement learning. In *Intelligent Systems Conference.*

Merton, R. C. (1969). Lifetime portfolio selection under uncertainty: The continuous-time case. *Journal of Economic Theory*, 3(4):373–413.

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.

Samuelson, P. A. (1969). Lifetime portfolio selection by dynamic stochastic programming. *The Review of Economics and Statistics*, 51(3):239–246.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347.*

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction.* MIT Press, 2nd edition.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *ICML.*